**German University in Cairo**
**Department of Computer Science**
**Assoc. Prof. Mervat Abu El-Kheir**

**Architecture of Massively Scalable Applications, Spring 2026**
**Lab 6 Manual: Authentication & Security**

Welcome to the Authentication Lab! In Labs 1–3 you built REST APIs and containerized them. In Labs 4–5 you connected Spring Boot to six databases. In this lab you will secure your application with authentication and authorization. You will implement three approaches — session-based auth, Redis-backed sessions, and JWT (JSON Web Tokens) — and integrate them with Spring Security's filter chain. By the end of this session you will have a fully secured Spring Boot API with role-based access control.

# 1 Project Setup

The starter project contains a Spring Boot skeleton with PostgreSQL, Spring Security, and the JJWT library pre-configured.

Clone the repo that has the base code to start working on this lab manual through this link `https://github.com/Scalable-2026/Lab06-Base-Code.git`

a) Open a terminal in the project root directory and start the database:

```
docker compose up -d
```

Verify the container is running:

```
docker ps
```

You should see a `postgres` container on port 5432.

b) Open the project in IntelliJ IDEA and set the SDK to **JDK 25**.

c) Open `pom.xml` and confirm the following dependencies are present:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
```

The `spring-boot-starter-security` dependency locks down **all** endpoints by default — every request returns 401 until you configure security rules.

d) Open `src/main/resources/application.yml` and confirm the connection properties:

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/lab06db
    username: postgres
    password: postgres
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

jwt:
  secret: dGhpcyBpcyBhIHNlY3JldCBrZXkgZm9yIEpXVCBzaWduaW5n
  expiration: 3600000 # 1 hour in milliseconds
```

The `jwt.secret` is a Base64-encoded key used to sign JWT tokens. The `jwt.expiration` controls how long tokens remain valid. In production, these values should come from environment variables — never hardcode secrets in source code.

e) The starter code includes a `User` entity (with id, name, email, password, role, and createdAt fields), a `Role` enum (`ADMIN`, `USER`), and a `UserRepository` with `findByEmail()` and `existsByEmail()` methods. We will examine these in detail in Section 7.

f) Run the application and hit `http://localhost:8080/api/health`. If Spring Security is active, you will get a 401 response — this is expected. We will configure public endpoints shortly.

# 2 Password Hashing with BCrypt

Passwords must **never** be stored as plain text. If an attacker gains access to your database, plain-text passwords are instantly compromised. Hashing transforms the password into an irreversible string. BCrypt is the standard for password hashing because it is intentionally slow (configurable work factor) and includes a built-in salt.

## 2.1 BCrypt Configuration

a) Open the `SecurityConfig` class in the `config` package. It already includes a `BCryptPasswordEncoder` bean:

```java
@Bean
public BCryptPasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

This exposes BCrypt as a Spring Bean that can be injected anywhere in the application. We will expand this class with the full security configuration in Section 6.

b) BCrypt is used in two places:

```java
// Registration -- hash the password before saving
String hashed = encoder.encode("myPassword");

// Login -- verify the password against the stored hash
boolean matches = encoder.matches("myPassword", hashed);
```

`encode()` produces a different hash string every time you call it (because of the random salt), but `matches()` always works because BCrypt embeds the salt *inside* the hash string itself. There is no need for a separate salt column in the database.

c) **Never use MD5 or SHA for passwords.** MD5 can compute billions of hashes per second, making brute-force attacks trivial. BCrypt is intentionally slow (10+ rounds by default), which makes brute-force computationally expensive.

# 3 Session-Based Authentication

Session-based authentication is the traditional approach. The server stores session data and sends the client a cookie containing the session ID. Every subsequent request includes this cookie automatically.

## 3.1 How Sessions Work

a) Browser sends `POST /login` with email and password.

b) Server validates credentials and creates a Session (e.g., ID: `abc123`) in the Session Store.

c) Server responds with `Set-Cookie: JSESSIONID=abc123`.

d) Browser sends `GET /api/users` with `Cookie: JSESSIONID=abc123`.

e) Server looks up session `abc123` in the Session Store, identifies the user, and returns data.

Spring manages this lifecycle automatically — the `JSESSIONID` cookie is sent and received transparently.

## 3.2 Login with HttpSession

a) Open the `AuthController` class and implement the login endpoint using `HttpSession`:

```
@PostMapping("/api/auth/login")
public ResponseEntity<String> login(
        @RequestBody LoginRequest request,
        HttpSession session) {
    User user = authService.authenticate(
        request.email(), request.password());
    session.setAttribute("userId", user.getId());
    session.setAttribute("role", user.getRole());
    return ResponseEntity.ok("Login successful");
}
```

`HttpSession` is injected as a method parameter. `setAttribute()` stores key-value pairs in the session. Spring automatically creates the session and sends the `JSESSIONID` cookie — no manual cookie handling needed.

**Note:** The session-based code in this section illustrates the traditional pattern. We will not implement the `authenticate()` method — in Section 8, we build the JWT-based `login()` method instead.

## 3.3 Protecting Endpoints with Session Checks

a) Implement a protected endpoint that checks the session:

```
@GetMapping("/api/users/profile")
public ResponseEntity<User> getProfile(
        HttpSession session) {
    Long userId = (Long) session.getAttribute("userId");
    if (userId == null) {
        return ResponseEntity.status(401).build();
    }
```

```
    User user = userService.findById(userId);
    return ResponseEntity.ok(user);
}
```

`getAttribute()` returns `null` if the session key does not exist. This manual null check is a manual authentication check — tedious because it must repeat in every protected endpoint. Spring Security automates this (coming in Section 6).

### 3.4 Logout

a) Implement the logout endpoint:

```
@PostMapping("/api/auth/logout")
public ResponseEntity<String> logout(
        HttpSession session) {
    session.invalidate();
    return ResponseEntity.ok("Logged out");
}
```

`invalidate()` removes all session attributes, marks the session ID as invalid, and the browser cookie becomes useless. Always use `POST` for logout, not `GET`.

### 3.5 Session Configuration

a) Add session configuration to `application.yml`:

```
server:
  servlet:
    session:
      timeout: 30m      # session expires after 30 min
      cookie:
        name: JSESSIONID
        http-only: true # JS cannot access cookie
        secure: true    # only sent over HTTPS
        same-site: Lax  # CSRF protection
```

`http-only` prevents XSS attacks from stealing session cookies via `document.cookie`. `secure` ensures the cookie is only sent over HTTPS. `same-site: Lax` provides basic CSRF protection.

### 3.6 Sessions Don't Scale

Sessions are stored in server memory. With a load balancer and multiple servers:

- Request 1 hits Server 1 — session is created on Server 1.
- Request 2 hits Server 2 — session is NOT found, user gets 401.

Sticky sessions (routing a user always to the same server) are a band-aid: they create single points of failure, uneven load distribution, and deployment nightmares. The real solution is externalizing the session store.

## 4 Scaling Sessions with Redis

Redis stores all data in RAM. By using Redis as a shared session store, all servers can read and write sessions from the same place. Any server can handle any request.

## 4.1 Spring Session + Redis Setup

a) Add the dependencies to `pom.xml`:

```
<dependency>
    <groupId>org.springframework.session</groupId>
    <artifactId>spring-session-data-redis</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

b) Add the Redis configuration to `application.yml`:

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
  session:
    store-type: redis
```

That is it — two dependencies and three lines of config. Spring Session intercepts `HttpSession` automatically.

c) Your existing code works as-is:

```
session.setAttribute("userId", user.getId());
// This now writes to Redis instead of server memory
// No code changes needed!
```

`setAttribute()` now writes to Redis, `getAttribute()` reads from Redis, and `invalidate()` deletes from Redis. The controller code is identical — zero changes.

d) Add a Redis service to `docker-compose.yaml`:

```
services:
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
  app:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      - redis
    environment:
      SPRING_DATA_REDIS_HOST: redis
volumes:
  redis-data:
```

### 4.2 When Redis Sessions vs JWT

| Choose Redis Sessions When... | Choose JWT When... |
|---|---|
| Building a traditional web app | Building microservices |
| Need instant session revocation | Mobile app authentication |
| Already using Redis for caching | Stateless API is a priority |
| Server-rendered pages (SSR) | Cross-domain / third-party auth |

Sections 3 and 4 demonstrated session-based authentication — the traditional approach. Sessions require server-side storage and do not scale horizontally without an external store like Redis. From this point forward, we switch to JWT — the stateless approach used by microservices and modern APIs. The session code above was for understanding the fundamentals; the rest of this lab builds the JWT-based system that your project will use.

## 5 Token-Based Authentication: JWT

JWT (JSON Web Token) is the stateless alternative to sessions. Instead of storing session data on the server, the server issues a signed token that the client stores and sends with every request. The token itself contains all the information needed to authenticate the user — no server-side storage required.

### 5.1 JWT Structure

A JWT has three parts separated by dots: `header.payload.signature`

- **Header:** Algorithm and token type (e.g., HS256, JWT).
- **Payload:** Claims — the data (subject, role, expiration, issued-at).
- **Signature:** HMAC-SHA256 of the header + payload, signed with the secret key. This prevents tampering — if anyone modifies the payload, the signature will not match.

### 5.2 JWT Flow (7 Steps)

a) Client sends `POST /api/auth/login` with email and password.

b) Server validates credentials against the database.

c) Server creates a JWT containing the user's email and role, signed with the secret key.

d) Client receives the JWT and stores it (typically in localStorage for SPAs).

e) For every subsequent request, the client includes the token in the `Authorization` header: `Bearer eyJhbG...`

f) Server verifies the token's signature without any database lookup.

g) If valid, server processes the request and returns data.

After login, the server never needs to look up session data — the token IS the proof.

### 5.3 JWT Configuration

a) The JWT secret and expiration are already in `application.yml`:

```
jwt:
  secret: dGhpcyBpcyBhIHNlY3JldCBrZXkgZm9yIEpXVCBzaWduaW5n
  expiration: 3600000 # 1 hour in milliseconds
```

b) Open `JwtConfig` in the `config` package and add the `@Value` bindings:

```
@Component
public class JwtConfig {
    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private long expiration;

    // getters
}
```

`@Value` injects values from `application.yml` into the config class. In production, override with environment variables: `JWT_SECRET=...` in `docker-compose.yml`. Spring Boot automatically maps the `JWT_SECRET` environment variable to the `jwt.secret` property.

## 5.4 Creating JWTs with JJWT

a) Open `JwtService` in the `service` package. The class injects `JwtConfig` via constructor injection:

```
@Service
public class JwtService {

    private final JwtConfig jwtConfig;

    public JwtService(JwtConfig jwtConfig) {
        this.jwtConfig = jwtConfig;
    }
```

b) Implement the `generateToken` method:

```
public String generateToken(User user) {
    return Jwts.builder()
        .subject(user.getEmail())
        .claim("role", user.getRole().name())
        .issuedAt(new Date())
        .expiration(new Date(
            System.currentTimeMillis()
                + jwtConfig.getExpiration()))
        .signWith(getSigningKey())
        .compact();
}

private SecretKey getSigningKey() {
    byte[] keyBytes = Decoders.BASE64.decode(
        jwtConfig.getSecret());
    return Keys.hmacShaKeyFor(keyBytes);
}
```

JJWT provides a fluent builder API:

- `subject()` sets the "sub" claim — typically the user's email.
- `claim()` adds custom claims like the user's role.
- `issuedAt()` records when the token was created.
- `expiration()` sets when the token becomes invalid. The value comes from `jwtConfig.getExpiration()` (configured in `application.yml`).
- `signWith()` uses the secret key to create the HMAC signature.

- `compact()` produces the final encoded JWT string.

`getSigningKey()` decodes the Base64-encoded secret from `jwtConfig.getSecret()` into a `SecretKey` object suitable for HMAC-SHA256.

### 5.5 Parsing and Validating JWTs

a) Implement the validation methods in `JwtService`:

```
public String extractUsername(String token) {
    return extractClaims(token).getSubject();
}

public boolean isTokenValid(String token) {
    try {
        extractClaims(token);
        return true;
    } catch (JwtException e) {
        return false;
    }
}

private Claims extractClaims(String token) {
    return Jwts.parser()
        .verifyWith(getSigningKey())
        .build()
        .parseSignedClaims(token)
        .getPayload();
}
```

`extractUsername()` gets the "sub" claim — the email set during token creation. `isTokenValid()` tries to parse the token; if parsing fails for any reason (expired, tampered, wrong key), it returns `false`. `extractClaims()` is the workhorse — `parseSignedClaims()` verifies the signature integrity AND checks expiration in one call.

### 5.6 Access Tokens vs Refresh Tokens

| Access Token | Refresh Token |
|---|---|
| Short-lived (15 min – 1 hour) | Long-lived (7 – 30 days) |
| Sent with every API request | Used ONLY to get new access tokens |
| Contains user claims (role, email) | Stored securely (httpOnly cookie) |
| If stolen, damage is time-limited | Can be revoked server-side |

This course focuses on access tokens. Refresh tokens add complexity but are essential in production.

## 6 Spring Security Filter Chain

Spring Security sits between HTTP requests and your controllers. It uses a chain of filters — each filter inspects or modifies the request. If any filter rejects the request, it never reaches the controller.

The filter chain looks like: HTTP Request → CSRF Filter → CORS Filter → JWT Auth Filter → Authorization Filter → Controller

### 6.1 SecurityConfig: The Configuration

a) Open `SecurityConfig` and update it with the full security configuration:

```java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    public SecurityConfig(JwtAuthFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

    @Bean
    public SecurityFilterChain filterChain(
            HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s
                .sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS))
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(
                    new HttpStatusEntryPoint(
                        HttpStatus.UNAUTHORIZED)))
            .authorizeHttpRequests(auth -> auth
                .dispatcherTypeMatchers(
                    DispatcherType.ERROR).permitAll()
                .requestMatchers("/api/auth/**",
                    "/api/health").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthFilter,
                UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    @Bean
    public BCryptPasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Breaking down each part:

- `@EnableWebSecurity` activates Spring Security's web support.
- `@EnableMethodSecurity` enables `@PreAuthorize` annotations on methods.
- `exceptionHandling` with `HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)` — returns 401 (not the default 403) when an unauthenticated request hits a protected endpoint.
- `dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()` — allows error responses (like 409 from duplicate registration) to pass through the security filter chain instead of being blocked.
- `SessionCreationPolicy.STATELESS` — do not create `HttpSession`; we use JWT instead.
- `requestMatchers("/api/auth/**", "/api/health").permitAll()` — login, register, and health check endpoints are public (no token needed).
- `anyRequest().authenticated()` — everything else requires a valid token.
- `addFilterBefore(jwtAuthFilter, ...)` — our custom JWT filter runs BEFORE Spring's default auth filter.

Explicit constructor injection — no Lombok.

## 6.2 JwtAuthFilter: The Custom Filter

a) Open `JwtAuthFilter` and implement the class:

```
@Component
public class JwtAuthFilter
        extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthFilter(JwtService jwtService,
            UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }
```

`OncePerRequestFilter` ensures the filter runs exactly once per request, even with internal request forwarding.

b) Implement the `doFilterInternal` method:

```
@Override
protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain chain) throws Exception {
    String header = request.getHeader("Authorization");
    if (header == null
            || !header.startsWith("Bearer ")) {
        chain.doFilter(request, response);
        return;
    }
    try {
          String token = header.substring(7);
          String username =
              jwtService.extractUsername(token);
          if (username != null
                  && jwtService.isTokenValid(token)) {
              UserDetails details = userDetailsService
                  .loadUserByUsername(username);
              UsernamePasswordAuthenticationToken auth =
                  new UsernamePasswordAuthenticationToken(
                      details, null,
                      details.getAuthorities());
              SecurityContextHolder.getContext()
                  .setAuthentication(auth);
          }
    } catch (Exception e) {
        // Invalid token -- continue unauthenticated
    }
    chain.doFilter(request, response);
}
```

Step by step:

1. Extract the `Authorization` header. If missing or not `Bearer`, skip — let the request continue without authentication.
2. `substring(7)` removes the "Bearer " prefix to get the raw token.
3. Extract the username from the token (the "sub" claim).

4. Validate the token: check signature and expiration.
5. If valid, load the full `UserDetails` from the database.
6. Create an authentication token and set it in the `SecurityContextHolder`.
7. The entire token processing block is wrapped in a `try-catch`. If the token is malformed or corrupted, the exception is caught and the request continues without authentication — Spring Security will return 401 via the `AuthenticationEntryPoint` we configured.
8. **Critical:** `chain.doFilter()` must ALWAYS be called — whether authenticated or not. It is placed outside the try-catch to guarantee execution.

### 6.3 UserDetailsService Implementation

a) Open `CustomUserDetailsService` and implement the interface:

```
@Service
public class CustomUserDetailsService
        implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(
            UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String email)
            throws UsernameNotFoundException {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() ->
                new UsernameNotFoundException(
                    "User not found: " + email));

        return new org.springframework.security.core
                .userdetails.User(
            user.getEmail(),
            user.getPassword(),
            List.of(new SimpleGrantedAuthority(
                "ROLE_" + user.getRole().name()))
        );
    }
}
```

**Important:** Spring Security's `User` class is different from your custom `User` entity. Spring's `org.springframework.security.core.userdetails.User` implements `UserDetails` and is what Spring Security uses internally. Your entity is the database model; Spring's User is the security model. The `loadUserByUsername` method bridges between the two.

Notice the `"ROLE_"` prefix: Spring Security expects authorities to follow this convention. Your enum `Role.ADMIN` becomes the authority string `ROLE_ADMIN`. This is required for `@PreAuthorize("hasRole('ADMIN')")` to work — the `hasRole()` method automatically prepends `ROLE_` when checking.

## 7 Role-Based Access Control (RBAC)

Different users need different levels of access. Roles group permissions into manageable categories.

## 7.1 The Role Enum and User Entity

a) Define the `Role` enum:

```
public enum Role {
    ADMIN,
    USER
}
```

b) Open the `User` entity and add the role field:

```
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String password;

    @Enumerated(EnumType.STRING)
    private Role role = Role.USER;

    // constructor, getters, setters
}
```

`@Enumerated(EnumType.STRING)` stores "ADMIN" or "USER" in the database column as a string, not as 0 or 1. Always use `STRING`.

## 7.2 Securing Endpoints with @PreAuthorize

a) Add role-based security to controller methods:

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping("/profile")
    public String getProfile() {
        return "User Profile";
    }

    @GetMapping("/admin/dashboard")
    @PreAuthorize("hasRole('ADMIN')")
    public String adminDashboard() {
        return "Admin Dashboard";
    }
}
```

`@PreAuthorize` is placed on **controller** methods, not service methods. Security is a cross-cutting concern — keep it in the controller layer (which handles HTTP and security), not in business logic. The service should never know about roles or permissions.

b) `@PreAuthorize` uses SpEL (Spring Expression Language) — a mini expression language built into Spring:

- `"hasRole('ADMIN')"` — basic role check.
- `"hasRole('USER') or hasRole('ADMIN')"` — either role.
- `"hasRole('ADMIN') and #id != 1"` — combined conditions.

The string evaluates to true or false at runtime. If false, Spring returns 403 Forbidden.

c) **Do not forget:** `@EnableMethodSecurity` must be present on your `SecurityConfig` class. Without it, `@PreAuthorize` annotations are silently ignored.

# 8 The Complete Login/Register Flow

## 8.1 DTOs: Request and Response Records

a) Create the DTOs in the `dto` package:

```
public record RegisterRequest(
    String name,
    String email,
    String password
) {}

public record LoginRequest(
    String email,
    String password
) {}

public record AuthResponse(
    String token,
    long expiresIn
) {}
```

Java records are perfect for DTOs: immutable, concise, automatic `equals`/`hashCode`/`toString`. No Lombok needed.

## 8.2 AuthController

a) Create `AuthController` with the register and login endpoints:

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(
            @RequestBody RegisterRequest request) {
        return ResponseEntity.ok(
            authService.register(request));
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(
            @RequestBody LoginRequest request) {
        return ResponseEntity.ok(
            authService.login(request));
    }
}
```

The controller is intentionally thin — it delegates all logic to the service layer. Explicit constructor injection, no Lombok.

## 8.3 AuthService: Register and Login

a) Create `AuthService` with both methods:

```
@Service
public class AuthService {
    private final UserRepository userRepository;
    private final BCryptPasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    public AuthService(UserRepository userRepository,
            BCryptPasswordEncoder passwordEncoder,
            JwtService jwtService) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtService = jwtService;
    }

    public AuthResponse register(RegisterRequest req) {
        if (userRepository.existsByEmail(req.email())) {
            throw new ResponseStatusException(
                HttpStatus.CONFLICT,
                "Email already exists");
        }
        User user = new User();
        user.setName(req.name());
        user.setEmail(req.email());
        user.setPassword(
            passwordEncoder.encode(req.password()));
        userRepository.save(user);
        String token = jwtService.generateToken(user);
        return new AuthResponse(token, 3600000);
    }

    public AuthResponse login(LoginRequest req) {
        User user = userRepository
            .findByEmail(req.email())
            .orElseThrow(() ->
                new ResponseStatusException(
                    HttpStatus.UNAUTHORIZED,
                    "Invalid credentials"));

        if (!passwordEncoder.matches(
                req.password(), user.getPassword())) {
            throw new ResponseStatusException(
                HttpStatus.UNAUTHORIZED,
                "Invalid credentials");
        }

        String token = jwtService.generateToken(user);
        return new AuthResponse(token, 3600000);
    }
}
```

Registration: check for duplicate email (409 Conflict if exists), create user, hash password with BCrypt, save, generate JWT. Login: find user by email, verify password with `matches()`, generate JWT. Both use `ResponseStatusException` which maps directly to HTTP status codes — no

`@ControllerAdvice` needed. Notice the generic "Invalid credentials" message for both email-not-found and wrong-password — never reveal which credential failed.[1]

### 8.4 Testing the Flow

a) Test with Postman or curl:

**Step 1 — Register:**

```
curl -X POST localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Mohamed","email":"m@guc.edu.eg",
      "password":"secret123"}'
```

**Step 2 — Login:**

```
curl -X POST localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"m@guc.edu.eg",
      "password":"secret123"}'
```

**Step 3 — Access Protected Endpoint:**

```
curl localhost:8080/api/users/profile \
  -H "Authorization: Bearer eyJhbG..."
```

Replace `eyJhbG...` with the actual token from the login response.

b) Verify error handling:

- **401 Unauthorized:** "Who are you?" — no token provided, token expired, or invalid token.
- **403 Forbidden:** "I know who you are, but you can't do this." — valid token but wrong role (e.g., USER trying to access an ADMIN endpoint).

## 9 Session vs JWT: Quick Reference

| Criteria | Session-Based | JWT |
|---|---|---|
| State | Stateful (server stores sessions) | Stateless (self-contained) |
| Storage | Server memory or Redis | Client-side |
| Scalability | Needs shared store | Any server can validate |
| Revocation | Easy (delete session) | Hard (wait for expiry) |
| Size | Small cookie (∼32 bytes) | Larger token (∼1KB) |
| Mobile | Cookie management is tricky | Easy — send in header |
| CSRF | Vulnerable (need CSRF token) | Not vulnerable |
| Performance | DB/Redis lookup per request | CPU-only verification |

- **Choose Sessions:** Traditional web app, instant revocation, server-rendered pages.
- **Choose JWT:** Microservices, mobile apps, cross-domain auth, horizontal scaling.
- **Choose Hybrid:** JWT access tokens with a Redis-based revocation blacklist.

[1] The slides show `RuntimeException` with separate error messages to illustrate the security risk. `ResponseStatusException` is the production approach — it handles HTTP status mapping automatically.

## 10 Lab Solution

You can clone this repo `https://github.com/Scalable-2026/Lab-06-Solution.git` to access the solution of the lab. You will also find a Postman collection that you can import in Postman and try the different routes.